

Tennis Match Analysis from a Single Broadcast Video

A Classical Computer-Vision Pipeline for Court Geometry, Player Tracking, and Shot Classification

Signal, Image and Video — Project Report • A.Y. 2025–2026 Marco Massa • Alfonso Antognozzi • Davide Cabitza

Abstract. This report presents an end-to-end system that turns a single fixed-camera tennis broadcast clip into structured match data: the court geometry, the continuous trajectories of both players, the ball track, and a shot-by-shot breakdown (forehand/backhand, flat/slice/dropshot/lob, and serve/smash). The court markings are reconstructed once via a white-line mask, Hough transform and a RANSAC homography, which makes every subsequent measurement metric. Players are segmented with a running-average background model and kept identity-stable through net crossings; the ball, being small and fast, is the only stage that uses a learned detector (YOLOv8). Shots are detected from the ball's kinematics near a player and classified with perspective-robust, scale-free features. The seven stages are integrated into a single headless pipeline with per-step error isolation, validated through reproducible self-tests (shot-type classification at $21/23 = 91.3\%$ on a hand-labelled rally) and a quantitative evaluation framework against manual ground truth. The system runs on commodity footage with no wearables and no training for the geometric stages.

1. Introduction

Broadcast tennis is rich in tactical information — where players stand, how far they run, which shots they hit — but that information is locked inside the pixels. Extracting it automatically usually relies on calibrated multi-camera rigs or wearable sensors. This project shows that a **single, uncalibrated broadcast clip** is enough to recover a useful match summary using mostly *classical* computer vision.

The guiding design principle is "**reconstruct the geometry once, then everything follows.**" A tennis court has a known, standardised size (ITF), so the four court corners and four service-box corners are sufficient to fit a homography from image pixels to real-world metres. Once that mapping exists, player positions become court positions, pixel displacements become speeds in km/h, and ball contacts become shots placed on a true top-down map.

The system recognises the full vocabulary of a rally:

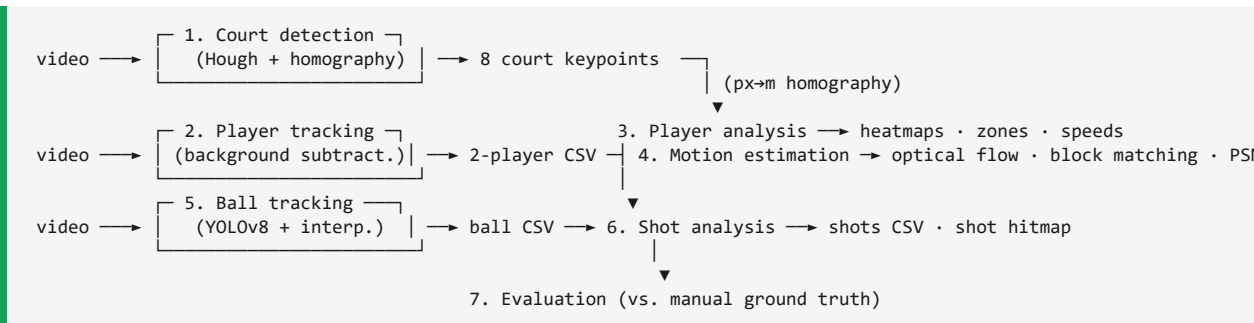
Layer	Classes	How it is decided
Stroke	forehand · backhand · unknown	side of the body the ball is struck on (perspective-aware)
Shot type	flat · slice · dropshot · lob	shape of the outgoing ball trajectory
Overhead	serve · smash	ball struck above the head, by context (toss vs. descending ball)

Only the ball detector is learned; court detection, player tracking, motion estimation and shot reasoning are deterministic, interpretable and tunable.

The test sequence used throughout this report (`Input_video2`) is a broadcast clip of an Alcaraz–Sinner rally (Miami 2023), 1008 frames at 30 fps (~33.6 s).

2. System Overview

The pipeline is organised as **seven sequential stages**. Each stage consumes the artefacts of the previous ones and writes its own CSV/figures, so any stage can be run, inspected or re-run in isolation.



#	Stage	Method (classical unless noted)	Output
1	Court detection	white mask + Hough + ITF filter + RANSAC homography	<code>*_court.csv</code> (8 keypoints)
2	Player tracking	running-average background + min-cost association	<code>players_*.csv</code>
3	Player analysis	homography → metres, statistics	heatmaps, zones, speed stats

#	Stage	Method (classical unless noted)	Output
4	Motion estimation	optical flow + block matching	flow fields, PSNR
5	Ball tracking	YOLOv8 + two-pass + interpolation	ball_*.csv
6	Shot analysis	ball kinematics + classifiers	shots.csv , shot_hitmap.png
7	Evaluation	IoU vs. annotated ground truth	error metrics

3. Court Detection and Geometry

3.1 Detecting the lines

The court lines are the only fixed reference in the scene, so they are detected first — directly from the white markings, without any training. The pipeline is:

1. **White HSV mask** to isolate the painted lines.
2. **Probabilistic Hough transform** (`threshold=150` , `minLineLength=80` ,

`maxLineGap=30`) to extract straight segments.

1. **Intersection clustering** of the segments (deterministic, `radius=50 px` , centroid averaging) to obtain candidate corner points.

1. **ITF-proportion filtering**: the doubles alley and the net are rejected by matching detected line spacings to the known singles/doubles ratio (`ALLEY_RATIO ≈ 0.125`).

1. **Far-line recovery**: the far baseline is often thin and washed-out in broadcast footage, so it is recovered from a 1-D white-coverage profile and a projective-consistency check rather than from the Hough lines alone.

The output is a CSV of **eight labelled keypoints** — the four court corners (`TL` , `TR` , `BL` , `BR`) and the four service-box corners (`STL` , `STR` , `SBL` , `SBR`).

3.2 Pixels → metres

A single **RANSAC homography** (`CourtConverter`) maps the eight image keypoints onto the standard ITF singles court:

```
H = findHomography(pixel_keypoints, ITF_metres, method=RANSAC, reproj=3px)
(x_m, y_m) = H · (x_px, y_px, 1)

ITF singles court : 8.23 m × 23.77 m   (27 ft × 78 ft)
Service lines     : 18 ft from each baseline
```

RANSAC makes the fit robust to a single mislabelled keypoint, and points falling near the horizon line are guarded against division-by-zero (returned as `NaN` instead of $\pm\infty$). This homography is the backbone of the whole project: every metric quantity downstream is produced through it.

4. Player Tracking

Players are the moving foreground of an otherwise static scene, so they are segmented with a **running-average background model** and tracked by association:

- a running-average background ($\alpha = 0.15$) plus a static reference built from

the first warm-up frames;

- frame differencing $\rightarrow$ intensity threshold (`DIFF_THRESH = 15` , decoupled from

the frame rate) $\rightarrow$ morphological cleaning;

- connected components with a minimum-area gate (`MIN_COMPONENT_AREA = 300 px2`);
- a **2×2 minimum-cost association** that assigns the two current detections to

the two previous identities by minimising total displacement (with a `MAX_MOVE` gate), rather than greedy nearest-neighbour.

The min-cost association is what keeps **P1 and P2 from swapping identities** when they cross near the net — a greedy matcher flips the labels there, whereas the 2×2 cost comparison preserves them. Identity loss/re-acquisition is logged.

Output CSV columns: `frame, player_id, x, y, w, h, cx, cy, area` .

5. Player Analysis

Each player's **feet point** (`cx, y+h`) is projected to metres and aggregated into movement statistics and court-coverage maps. Instantaneous displacements above 45 km/h are discarded as tracking glitches.

- **Speeds** (mean / median / p95 / max, km/h) and total distance per player.
- **Court zones**: a 6×4 grid (six depth bands $\times$ four lateral columns) with

per-zone occupancy time and percentage.

- **Heatmaps**: Gaussian-smoothed position density per player and combined

(P1 warm, P2 cool), over the full walkable area including run-off.

- An animated top-down **minimap** of both players.

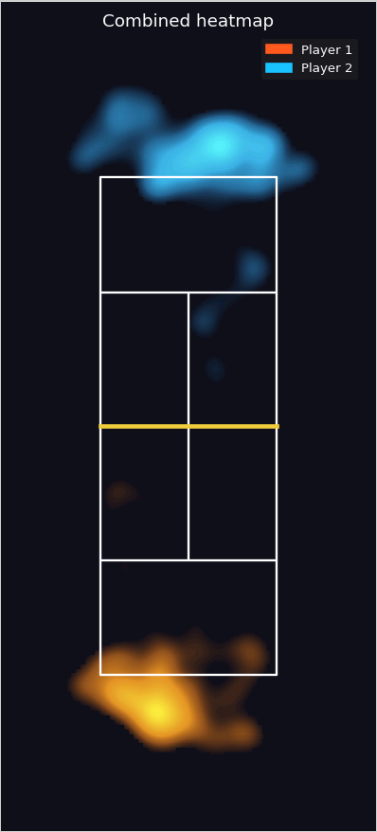


Figure 1 — Combined position heatmap (P1 warm, P2 cool). The near player covers the baseline corners; the far player is compressed by perspective at the top.

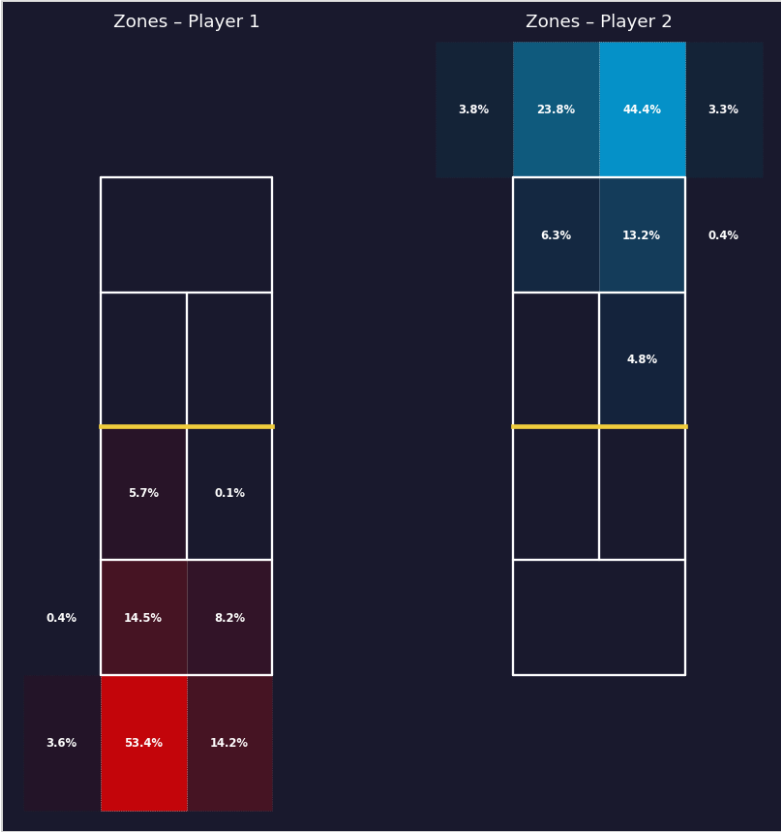


Figure 2 — Court-zone occupancy. Time spent in each of the 6×4 bands, overlaid on the court.

6. Ball Tracking

The ball is tiny, fast and frequently motion-blurred — the one stage where a learned detector is justified:

- a **YOLOv8** detector (`ball_tracker.pt` , confidence `0.65`);
- a **two-pass** scheme with frame-difference motion validation, which rejects

static false positives (line marks, logos);

- **interpolation** of short gaps, with an `interpolated` flag so downstream

stages know which samples are synthetic.

Output CSV: `frame, cx, cy, w, h, area, interpolated` .

7. Shot Detection and Classification

7.1 Detecting contacts

A shot is a **sudden change in the ball's vertical motion right next to a player**. On the Savitzky–Golay-smoothed ball track, a contact candidate is:

```
sign reversal of vertical velocity  v_y    (|v_y| ≥ 0.5 px/frame)    OR  
peak of the acceleration magnitude |a|    (|a| ≥ 1.5 px/frame²)  
AND the ball lies inside the player's expanded bounding box
```

Bounces also reverse the ball, but they happen *away* from the players, so the proximity gate removes them. Candidates belonging to the same contact are merged (min-gap clustering), keeping the strongest acceleration.

7.2 Stroke: forehand vs. backhand

At the contact frame, the ball's horizontal offset from the player's body axis decides the stroke, with a dead-band around the axis labelled `unknown` . Crucially the decision is **perspective-aware**: the near player is seen from behind (their right is image-right) while the far player faces the camera (their right is image-left), and left-handers invert the mapping.

7.3 Shot type: flat / slice / dropshot / lob

How the ball was struck is read from the **shape of the outgoing trajectory**. Because the same physical shot appears $\sim 3\times$ faster in pixels for the near player than the far one, raw pixel speed is never compared across sides. Perspective is neutralised with **scale-free features**:

- `pace` = pixel step-speed normalised by the ball's apparent size;
- `diefrac` = how quickly the ball decelerates (2nd-half / 1st-half path length);
- `reach` and `bowback` = penetration depth and how much the ball arcs back.

A short rule set then assigns flat / slice / dropshot / lob. The thresholds were fit on a 23-shot hand-labelled rally and are all exposed as CLI flags for per-camera retuning.

7.4 Overhead: serve and smash

Serve and smash are **overhead** shots — the ball is struck above the head — and are kept in a **separate overhead column**, so they never disturb the flat/slice/dropshot/lob labels. The common gate is geometric (ball clearly above the box top and horizontally aligned with the player). The two are then separated by the **vertical direction of the incoming ball**, a scale-free cue that is robust to the homography distorting an airborne ball near the baseline:

- **serve** — first shot of the clip, player stationary behind their own

baseline, and the incoming ball *rising* (the toss goes up);

- **smash** — the incoming ball *descending* (a lob coming down), struck downward

with a low apex.



Figure 3 — Shot hitmap: every shot placed at the player's court position at contact, coloured by shot type. P1 (circles) plays from the near baseline, P2 (triangles) from the far side; the cyan circle is the opening serve.

```
shots.csv columns: frame, time_s, player_id, side, hand, stroke, shot_type, overhead,
ball_pace, ball_cx, ball_cy, player_cx, player_x_m, player_y_m, ball_x_m, ball_y_m.
```

8. Motion Estimation

Two classic motion-estimation techniques — the core SIV material — cross-check player speed and quantify inter-frame motion.

- **Optical flow.** Dense **Farneback** flow and sparse **Lucas–Kanade** tracking

of Shi–Tomasi corners. The mean flow inside each player box is converted to km/h via the homography and compared with the position-based speed.

- **Block matching.** Sum-of-absolute-differences over fixed blocks, with both **three-step** and full search, motion compensation, and a **PSNR** gain report versus the no-motion baseline.

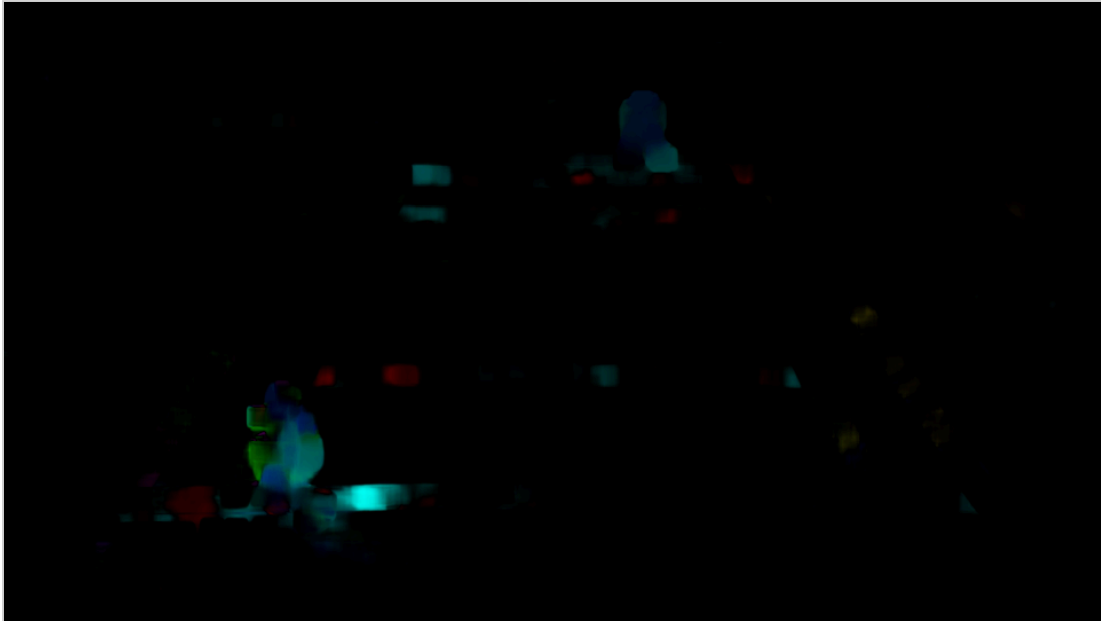


Figure 4 — Dense optical flow (Farneback): hue encodes direction, brightness encodes magnitude. The moving players and ball stand out against the static court.

9. Evaluation

Correctness is measured against **manual ground truth**, not eyeballed. A small annotation tool produces ground-truth player boxes and court keypoints, and the evaluator reports:

- **IoU** matching between predicted and ground-truth boxes (precision / recall);
- **centre error** (pixels) and **feet error** (metres, via the homography);
- **ID switches** as a measure of identity stability;
- **court keypoint error** versus annotated corners.

In addition, the shot module ships **reproducible self-tests** that double as regression guards:

Self-test	What it checks	Result
<code>--self-test</code>	synthetic rally, stroke classification	4/4
<code>--type-self-test</code>	23-shot hand-labelled rally, shot type	21/23 = 0.913
<code>--overhead-self-test</code>	synthetic serve/smash vs. groundstroke/lob	4/4

These tests pin the behaviour of the classifiers so later changes cannot silently degrade them.

10. Unified Pipeline and Live Viewer

`pipeline.py` runs all seven stages from a single command and is **headless by default** (`--display` enables OpenCV windows). It is built for robustness:

- the input frame rate is **auto-read** from the video;
- every step is wrapped in its own error guard, so one failure (a missing model,

a bad path) is reported and skipped instead of aborting the whole run;

- all paths are derived from the video name, and any stage can be skipped with

```
--skip-*
```

- a closing summary lists which steps **succeeded** / **were skipped** / **failed**.

```
python pipeline.py --video data/Input_video2.mp4
```

`live_view.py` replays the result in a single window: the video with player and ball overlays, a synced top-down minimap, a live shot-statistics panel, and an end-of-clip summary (heatmap · zones · hitmap).

11. Project Evolution

The project was built **bottom-up** over seven weeks, from isolated detectors to one integrated, evaluated pipeline. The git history (44 commits, three contributors, 2026-05-07 → 2026-06-21) groups naturally into five phases.

Phase	Period	Milestones
1 — Foundations	May 7–16	First court tracking (edge/Hough experiments), YOLO ball tracking, player tracking, mask debugging windows.
2 — Geometry	May 20–30	Ball-track CSV export; creation of <code>utils/</code> and the pixel→metre converter — the homography backbone that made all later metric analysis possible.
3 — Refactor & architecture	Jun 10–19	Repository restructure, large bug-fix passes, a complete annotated output video, and stabilisation of player tracking.
4 — Shot intelligence	Jun 19–20	Shot detection , forehand/backhand, the flat/slice/dropshot/lob classifier, ball-track interpolation, and finally serve/smash + the shot hitmap .
5 — Visualisation & delivery	Jun 21–22	The live viewer with live statistics, intensified heatmaps, the unified pipeline , and documentation.

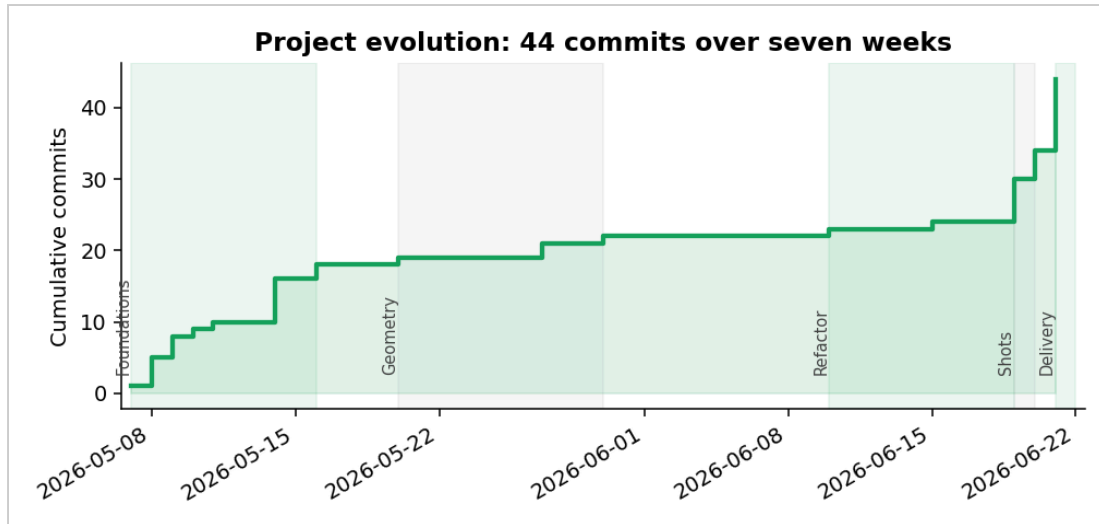


Figure 6 — Cumulative commits over the project's lifetime. The shaded bands mark the five development phases, from the early detectors to the integrated pipeline.

Two architectural decisions shaped everything that followed. **First**, isolating the pixel→metre homography in `utils/` (Phase 2) turned a set of pixel-space scripts into a metric measurement system. **Second**, the Phase-3 refactor split the monolith into independent, CSV-connected stages, which is what later allowed shot analysis (Phase 4) and the unified pipeline (Phase 5) to be added without destabilising the existing detectors.

The most recent work (Phase 4–5) also reflects an explicit *no-regression* discipline: new shot categories (serve/smash) were added as a **separate column** and guarded by self-tests, precisely so that adding rarer shots could not damage the already-correct groundstroke labels.

12. Results

On the `Input_video2` test clip the pipeline produces a complete, coherent match summary end-to-end:

- **Court:** 8 keypoints detected and a homography fitted with low (~ 1 px)

line-fit residuals on the detected court lines.

- **Players:** both players tracked across the full clip with stable identities

through net crossings.

- **Shots: 26 contacts** detected — 25 groundstrokes plus the opening

serve — broken down by stroke (forehand/backhand) and type (18 flat, 4 slice, 2 dropshot, 1 lob), and placed on the shot hitmap (Figure 3).

- **Shot-type classifier: 21/23 (91.3%)** against the hand-labelled rally,

stable across a wide threshold range.

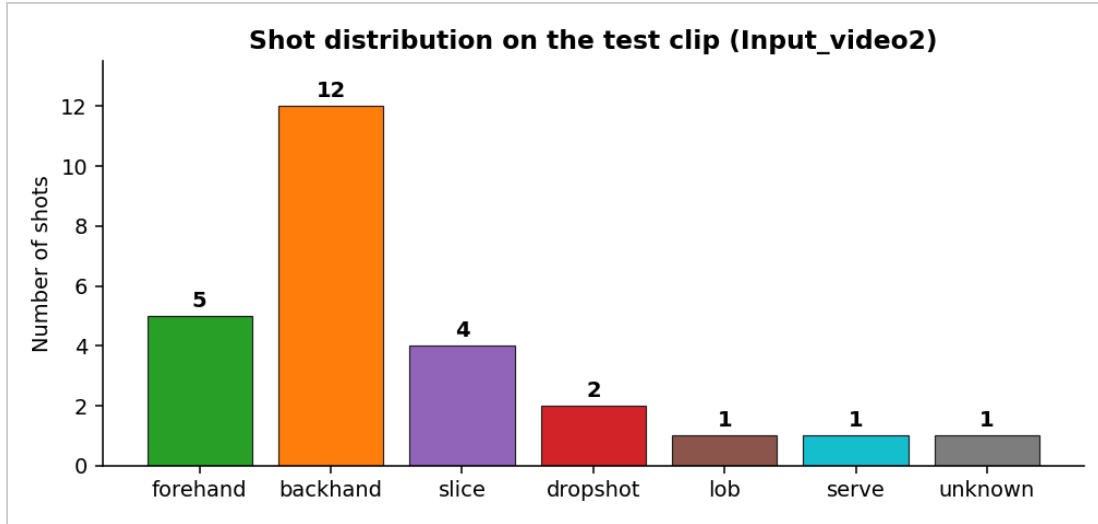


Figure 5 — Shot distribution on the test clip, by combined category (forehand/backhand groundstrokes, slice, dropshot, lob, and the opening serve).

The qualitative outputs (Figures 1–4) confirm the expected structure: the near player covers the baseline corners, the far player is perspective-compressed, the serve appears behind the near baseline, and the optical flow isolates the moving subjects from the static court.

13. Discussion and Limitations

Strengths.

- *Mostly training-free.* Only the ball uses a learned model; court, players,

motion and shot reasoning are deterministic and interpretable.

- *Metric by construction.* The single homography makes every downstream quantity physical (metres, km/h, court zones).

- *Perspective-robust shot reasoning.* Scale-free features (pace, diefrac, reach, bowback) and direction-based overhead cues avoid comparing pixel speeds across the foreshortened far half.

- *Engineered for safety.* Per-step error isolation, a stable CSV contract between stages, and self-tests that act as regression guards.

Limitations.

- *Single fixed camera.* The homography assumes a static broadcast viewpoint; a camera cut would require re-detection.

- *Ball-detector dependency.* Shot analysis quality is bounded by the YOLO ball track; long dropouts force interpolation and can blur a contact frame.

- *Airborne-ball geometry.* The court homography is a ground plane, so the

projected position of an airborne ball is only approximate — handled by using scale-free / direction features for shot reasoning rather than raw metres.

- *Overhead at clip boundaries.* A serve in the first frames depends on the

player tracker covering the clip start; a truncated or warm-up-only player CSV can miss it.

- *Thresholds tuned on one rally.* The shot-type cuts are fit on a single

hand-labelled clip and may need per-camera retuning (all are exposed as flags).

14. Conclusions and Future Work

This project shows that a single, uncalibrated tennis broadcast clip can be turned into structured match data with a mostly-classical computer-vision pipeline. By reconstructing the court geometry once and propagating it through every stage, the system delivers player tracking, movement statistics, motion estimation and a shot-by-shot breakdown — including serve and smash — from commodity footage, with no wearables and reproducible validation.

Future directions:

- **Camera-cut handling** and automatic re-detection of the court across shots.
- **Learned shot classification** trained on a larger labelled set, to replace

hand-tuned thresholds while keeping the interpretable features.

- **Rally / point segmentation** built on top of the detected serves.
- **Temporal models** for tracking and shot detection to better handle ball

dropouts and occlusions.

- **Broader validation** on multiple matches, players and broadcast styles.

Repository artefacts referenced in this report are produced by `pipeline.py` (stages 1–7) and `live_view.py`, and stored under `outputs/`. Figures 1–4 are the actual outputs generated on the `Input_video2` test clip.